Sarvajanik College of Commerce and Computer Application

Software Requirements Specification

BuildAdvisor – AI-Powered PC Build Recommender for Gamers

Version 2.0

Team Members

Sr. No.	Name	Roll No.	Division	Contact No.	Email
1					
2					
3					
4					

1. Introduction

1.1 Purpose

This document specifies the software requirements for BuildAdvisor, an AI-powered web application that recommends a compatible PC build to gamers based on their budget and target games. It guides the design, development, and testing of the system, and serves as a shared reference between the project team and evaluators.

1.2 Scope

BuildAdvisor allows a user to enter a budget and a target game/performance goal (e.g. resolution and FPS) and receive a complete, compatible PC build recommendation with component-level reasoning generated by an AI model. Users can also input their existing PC specifications to receive targeted upgrade suggestions. The system fetches live component prices, checks compatibility across components, and maintains a benchmark/specification database that backs the recommendations.

1.3 Definitions, Acronyms and Abbreviations

Term	Description
SRS	Software Requirements Specification
FPS	Frames Per Second, a measure of gaming performance
GPU	Graphics Processing Unit
CPU	Central Processing Unit
PSU	Power Supply Unit
API	Application Programming Interface
DFD	Data Flow Diagram
ER Diagram	Entity Relationship Diagram
Gemini API	Google's generative AI API used to produce plain-language build explanations

1.4 Overview

Section 2 describes the overall product context. Section 3 analyzes the problem, existing alternatives, feasibility, and system behaviour through use case, DFD, activity, and sequence diagrams. Sections 4 and 5 list functional and non-functional requirements. Section 6 covers external interfaces, Section 7 the architecture, and Section 8 the

database design. Sections 9–16 cover business rules, error handling, testing, security, limitations, future scope, traceability, and appendix material.

1.5 SDLC Model

This project follows the Agile (Scrum-based) Software Development Life Cycle model. The system is built as a set of largely independent features (build generator, compatibility checker, price fetcher, upgrade advisor, AI-based explanation, save/share/rating), which are well suited to short, testable iterations. AI-integration work (prompt design and response handling for the Gemini API) typically needs iterative refinement based on testing, which fits Agile's feedback-driven approach better than a fixed-requirements model such as Waterfall. Agile also matches the college project timeline, where progress is demonstrated incrementally across review milestones.

- Sprint 1: Requirement analysis, database design, and UI wireframes
- Sprint 2: Budget-based build generator and compatibility checker
- Sprint 3: Live price comparison and upgrade path suggestions
- Sprint 4: Gemini API integration for build explanations
- Sprint 5: Save/share/rating features, testing, and deployment

2. Overall Description

2.1 Product Perspective

BuildAdvisor is a standalone web application built as a BCA capstone project. It is a new, self-contained system, not a replacement for an existing product. It follows a client-server model with a Streamlit-based frontend and a Flask backend that connects to a component database and external services for pricing and AI-generated explanations.

2.2 Product Features

- Budget-based build generator (entry-level, mid-range, high-end tiers)
- Game-specific performance targeting (e.g. 1080p 144fps in Valorant vs 1440p 60fps in Cyberpunk)
- Component compatibility checker (CPU-motherboard socket, PSU wattage, RAM type, case size)
- Price comparison across retailers for each recommended component
- Upgrade path suggestions for users with an existing PC
- Save/share build lists, and community-submitted build ratings

2.3 User Classes

User Class	Description
Gamers (Buyers)	New PC builders who enter a budget and target games to receive a complete build recommendation with estimated FPS and pricing.
Existing PC Owners	Users who input their current specifications to receive prioritized, budget-aware upgrade suggestions.
Admin	Maintains the component database, updates prices and benchmarks, and moderates community-submitted build ratings.

2.4 Operating Environment

- Client: Any modern web browser (Chrome, Edge, Firefox) on desktop or mobile
- Server: Python 3.x runtime with Flask, deployable on a standard web/app hosting service
- Database: SQLite for development, upgradable to MySQL for production
- External services: Gemini API for explanations; retailer APIs or web scraping for live component prices

2.5 Constraints

- Backend must be implemented in Python (Flask) as per the team's finalized technology stack
- Frontend must be implemented using Streamlit
- AI-generated explanations depend on Gemini API availability and rate limits
- Live pricing accuracy depends on the availability and structure of third-party retailer pages/APIs

2.6 Assumptions and Dependencies

- Users have basic familiarity with gaming performance terms (resolution, FPS)
- A stable internet connection is available for live price fetching and AI API calls
- The component/benchmark database is kept reasonably up to date by the Admin

3. System Analysis

3.1 Problem Statement

Gamers, especially first-time PC builders, often overspend or end up with mismatched/bottlenecked components because they lack the technical knowledge to evaluate CPU-GPU pairing, compatibility, or realistic FPS expectations for their target games. This leads to buyer's remorse, wasted budget, and reliance on scattered, sometimes outdated forum advice.

3.2 Existing System

Currently, gamers rely on generic online PC-build tools, YouTube guides, or forum threads to plan a build. These tools typically let a user pick components manually and check basic compatibility, but they do not tailor recommendations to a specific game/FPS target, do not explain why a component was chosen in plain language, and rarely reflect live, local (India-specific) pricing. As a result, users must cross-reference multiple sources and still risk picking an unbalanced build.

3.3 Proposed System

BuildAdvisor addresses these gaps by combining a budget and game-target input with an automated compatibility check, live price comparison, and an AI-generated plain-language explanation of the recommended build. It also supports existing PC owners by suggesting the most cost-effective upgrade path instead of a full rebuild.

3.4 Advantages of Proposed System

- Personalized recommendations based on both budget and target game/FPS, not just budget alone
- Plain-language AI explanations instead of a bare parts list
- Live price comparison across retailers to avoid overpaying
- Automated compatibility checks reduce the risk of a non-functional build
- Upgrade-path guidance for existing PC owners, saving unnecessary full-rebuild cost

3.5 Feasibility Study

Technical Feasibility

The proposed stack (Python, Flask, Streamlit, SQLite/MySQL, Gemini API) is well documented, open-source or free-tier, and matches the team's existing skill set, making the project technically feasible within the project timeline.

Economic Feasibility

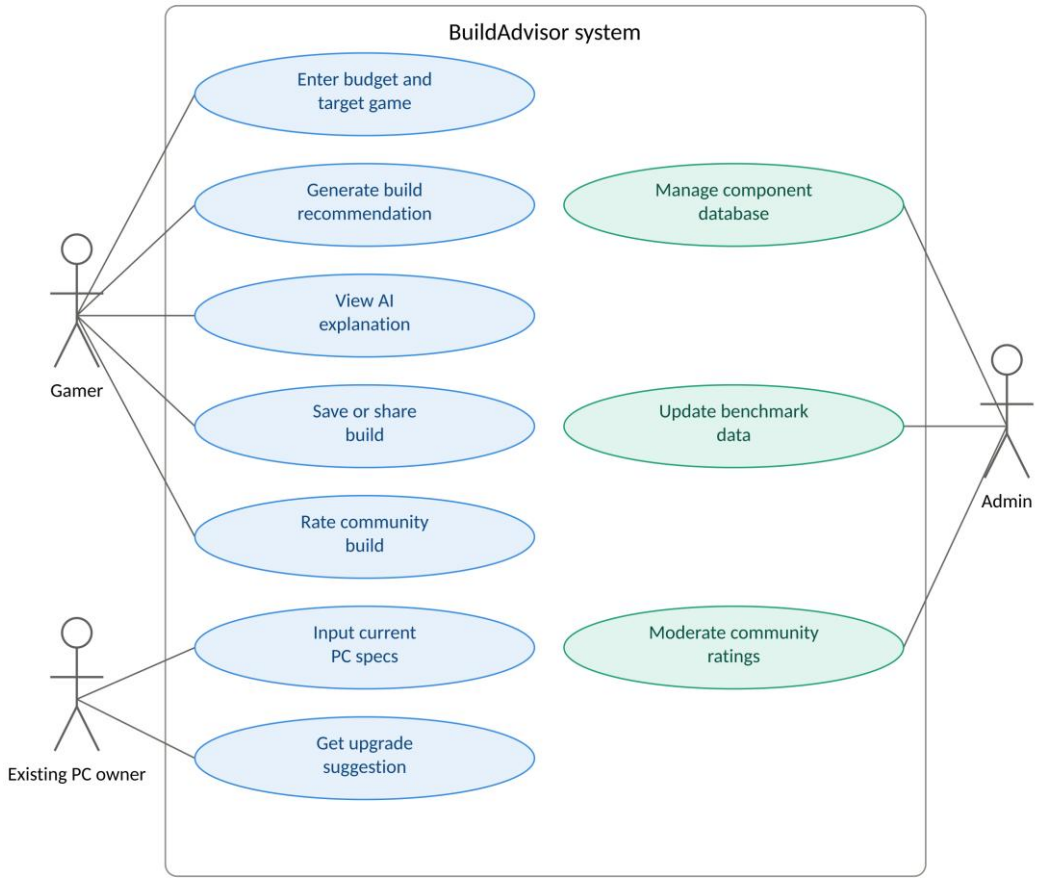
The system relies on open-source frameworks and free-tier API quotas, so development cost is minimal. The only ongoing cost, if deployed publicly, would be basic web hosting, making the project economically feasible for a student project.

Operational Feasibility

The interface is designed to be simple enough for gamers with no hardware background, requiring no special training. Since the target users are already familiar with similar online PC-build tools, adoption friction is expected to be low.

3.6 Use Case Diagram

The diagram below shows the three actors of the system — Gamer, Existing PC Owner, and Admin — and the use cases each interacts with inside the BuildAdvisor system boundary.



3.7 Detailed Use Cases

Field	Use Case: Generate Build Recommendation
Actor	Gamer
Precondition	User has a budget in mind and knows the target game(s)
Main Flow	1. User enters budget and target game. 2. System validates input. 3. System fetches matching components. 4. System checks compatibility. 5. System fetches live prices. 6. System requests an AI explanation. 7. System displays the recommended build.
Postcondition	A compatible, priced build with an explanation is shown to the user

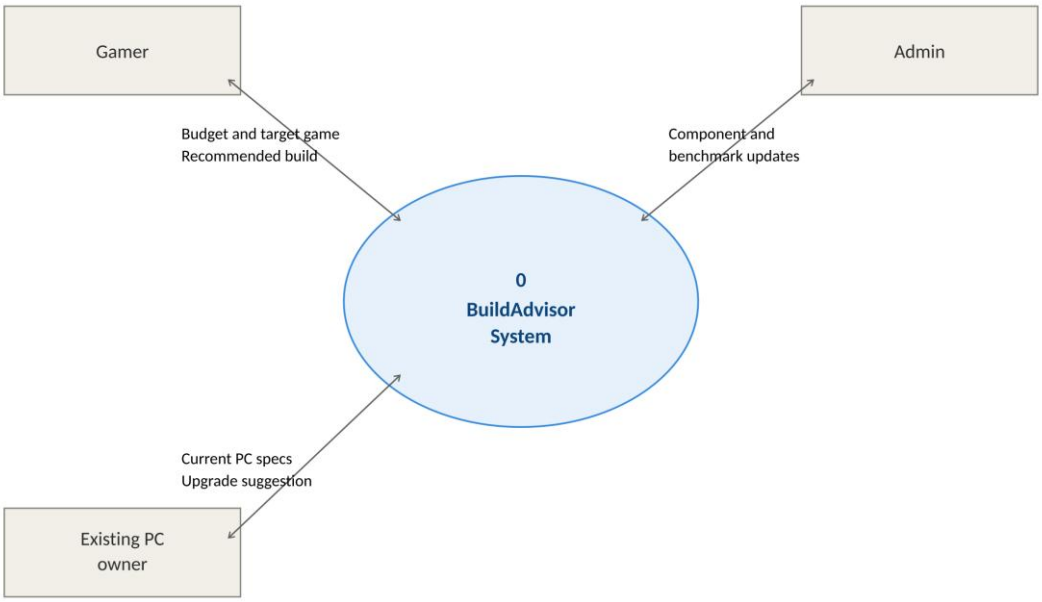
Field	Use Case: Get Upgrade Suggestion
Actor	Existing PC Owner
Precondition	User knows their current PC specifications

Field	Use Case: Get Upgrade Suggestion
Main Flow	1. User inputs current specifications and an optional additional budget. 2. System identifies the biggest performance bottleneck. 3. System recommends the most cost-effective upgrade.
Postcondition	User receives a prioritized upgrade recommendation

Field	Use Case: Manage Component Database
Actor	Admin
Precondition	Admin is authenticated
Main Flow	1. Admin adds, updates, or removes components. 2. Admin updates benchmark/FPS data. 3. Admin moderates community ratings.
Postcondition	Component/benchmark database and community ratings are up to date

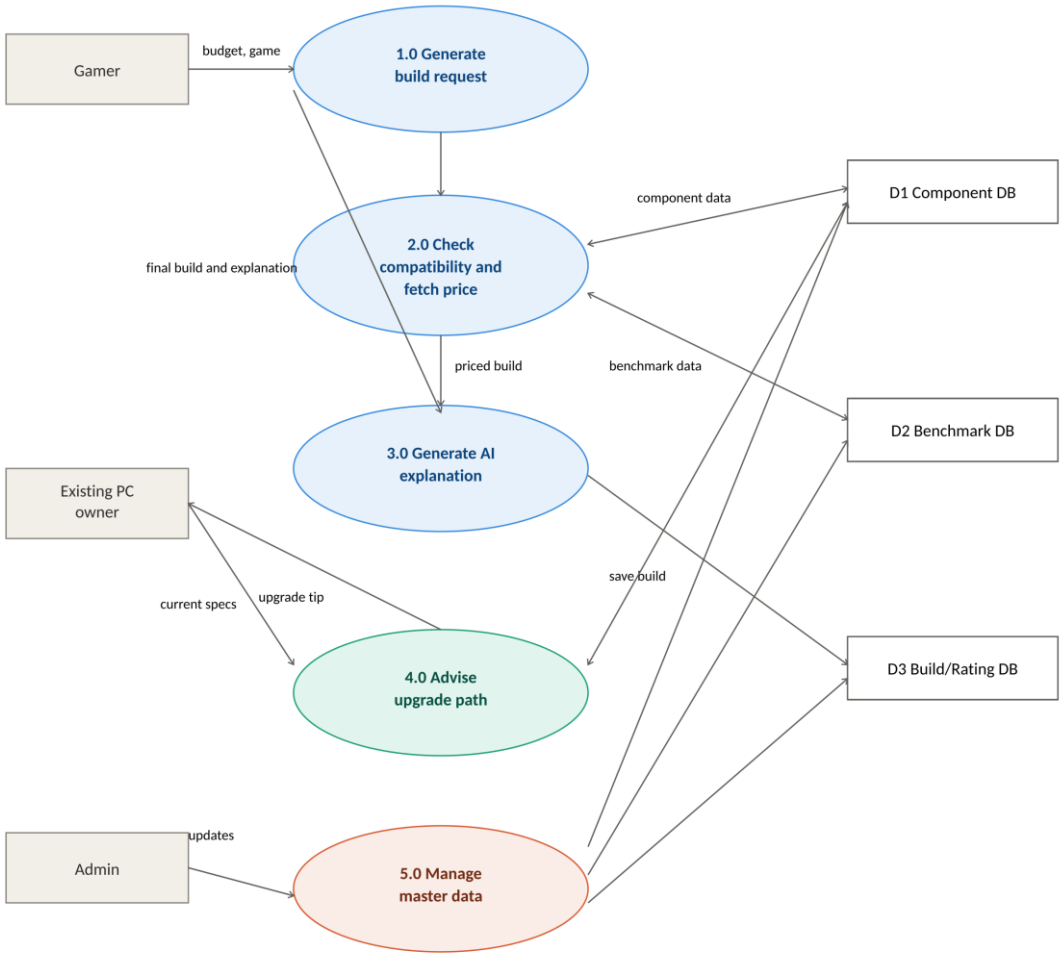
3.8 DFD Level 0 (Context Diagram)

The context diagram shows BuildAdvisor as a single process exchanging data with its three external entities.



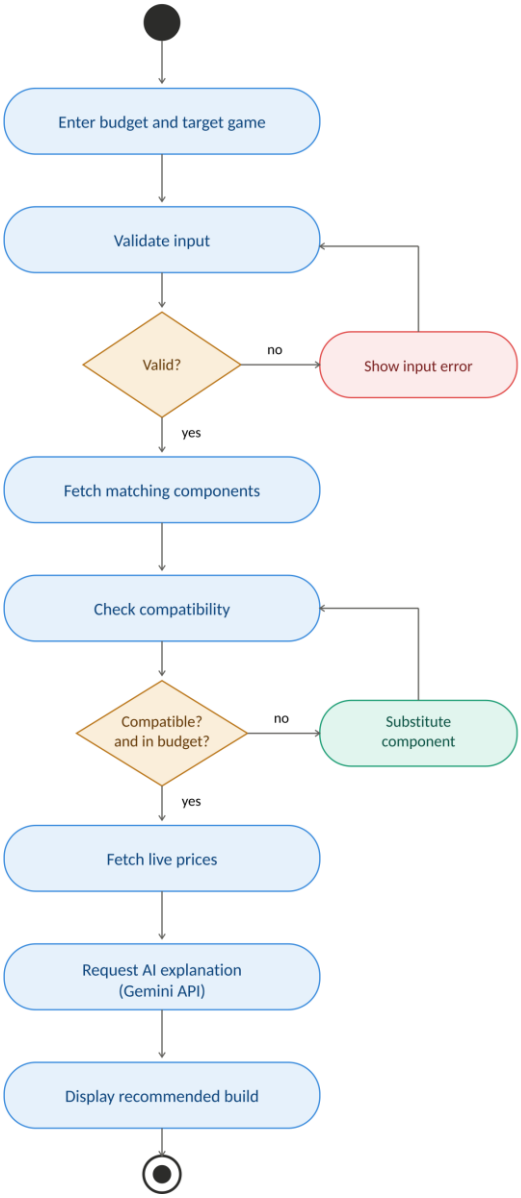
3.9 DFD Level 1

The level 1 diagram decomposes the system into five major processes and three data stores.



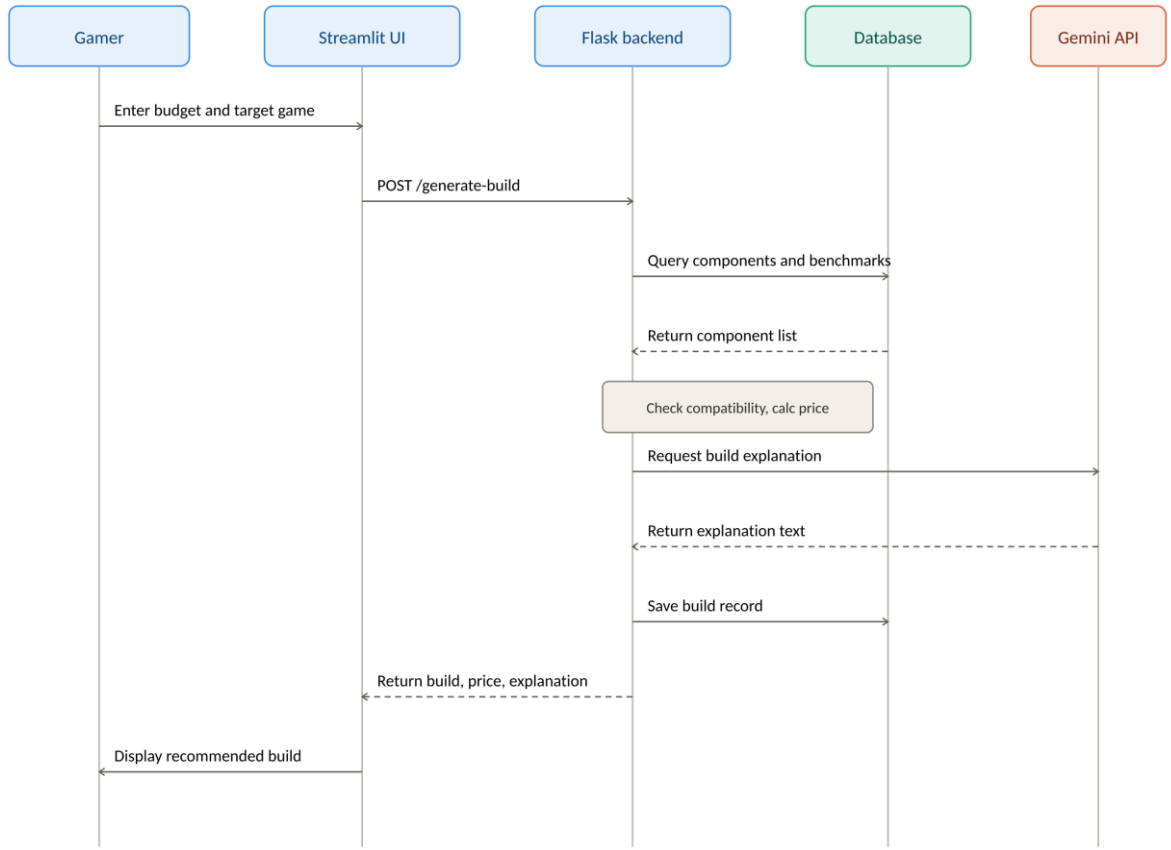
3.10 Activity Diagram

The activity diagram below traces the build-generation flow from user input to the final recommendation, including the validation and compatibility/budget decision points.



3.11 Sequence Diagram

The sequence diagram shows the interaction between the Gamer, the Streamlit frontend, the Flask backend, the database, and the Gemini API while generating a build recommendation.



4. Functional Requirements

4.1 Build Generation

The system shall generate a complete PC build (CPU, GPU, RAM, storage, motherboard, PSU, case) based on a user-specified budget range and tier (entry-level, mid-range, high-end).

1. The system shall accept a numeric budget input from the user.
1. The system shall classify the budget into a tier and select components within that price range.
1. The system shall ensure the total build cost does not exceed the specified budget.

4.2 Game Performance

The system shall allow the user to select target games and a desired resolution/FPS goal, and shall recommend components capable of meeting that target.

1. The system shall provide a list of supported games with resolution/FPS presets.
1. The system shall display an estimated FPS for the recommended build based on the benchmark database.

4.3 Compatibility

The system shall validate that all selected components are compatible with each other before presenting a build.

1. The system shall verify CPU and motherboard socket compatibility.
1. The system shall verify power supply wattage is sufficient for the selected CPU and GPU.
1. The system shall verify RAM type (DDR4/DDR5) matches motherboard support.
1. The system shall verify the selected components fit within the chosen case form factor.

4.4 Price Comparison

The system shall fetch and display current prices for each recommended component from multiple retailers.

1. The system shall retrieve price data via retailer APIs or web scraping.
1. The system shall display the lowest available price per component along with the retailer name.

4.5 Upgrade Advisor

The system shall allow an existing PC owner to input current specifications and receive prioritized upgrade recommendations.

1. The system shall accept current component specifications as input.
1. The system shall identify the component with the greatest performance bottleneck.
1. The system shall recommend the most cost-effective upgrade within a specified additional budget.

4.6 Gemini AI Explanation

The system shall use the Gemini API to generate a plain-language explanation of why each component was selected.

1. The system shall send the selected build and user constraints to the Gemini API.
1. The system shall display the returned explanation alongside the build.

4.7 Save, Share and Rate Builds

The system shall allow users to save a generated build, share it via a link, and rate community-submitted builds.

1. The system shall store saved builds against a user session or account.
1. The system shall generate a shareable identifier/link for a saved build.
1. The system shall allow users to submit a rating for a build.

4.8 Admin Management

The system shall provide an interface for the Admin to maintain the component and benchmark database.

1. The system shall allow the Admin to add, update, or remove components.
1. The system shall allow the Admin to update benchmark/FPS data.
1. The system shall allow the Admin to moderate (approve/remove) community ratings.

5. Non-Functional Requirements

5.1 Performance

- The system shall generate a build recommendation within 5 seconds under normal load, excluding external API latency.
- The system shall cache frequently requested price data to reduce repeated external calls.

5.2 Reliability

- The system shall handle failures of external services (pricing source, Gemini API) gracefully, showing a fallback message instead of crashing.
- The component database shall be backed up periodically to prevent data loss.

5.3 Usability

- The interface shall be usable by gamers with no technical hardware background.
- All recommendations shall include a plain-language explanation, not just a component list.

5.4 Security

- User-submitted data (saved builds, ratings) shall be validated before storage to prevent injection attacks.

- Admin functions shall be protected by authentication.

5.5 Maintainability

The codebase shall separate concerns between the Streamlit frontend, Flask backend, and data layer to allow independent updates to the component database, pricing logic, or AI integration.

5.6 Scalability

The backend shall be designed to remain stateless where possible so that additional users, components, or games can be added without major redesign. Database queries shall be indexed on frequently filtered fields (budget tier, component type, game).

5.7 Availability

The system shall aim for high availability during demonstration and normal use; when a third-party dependency (pricing source or Gemini API) is unavailable, the system shall continue to function using cached or last-known data rather than becoming fully unavailable.

6. External Interface Requirements

6.1 User Interface

The application shall provide a Streamlit-based web interface with: a build-configuration screen (budget slider, game/performance selector), a results screen (recommended build, estimated FPS, price breakdown, AI explanation), an upgrade-advisor screen, and an admin dashboard for database management.

6.2 Hardware Interface

No special hardware interfaces are required; the system runs on standard client and server hardware with internet connectivity.

6.3 Software Interface

Interface	Purpose
Flask (backend)	Handles business logic, request routing, and communication between the frontend and database/APIs
Streamlit (frontend)	Renders the user interface and captures user input
SQLite / MySQL	Stores component specifications, benchmark data, saved builds, and ratings
Gemini API	Generates plain-language explanations for recommended builds
Retailer APIs / web scraping (BeautifulSoup, requests)	Fetches live component pricing

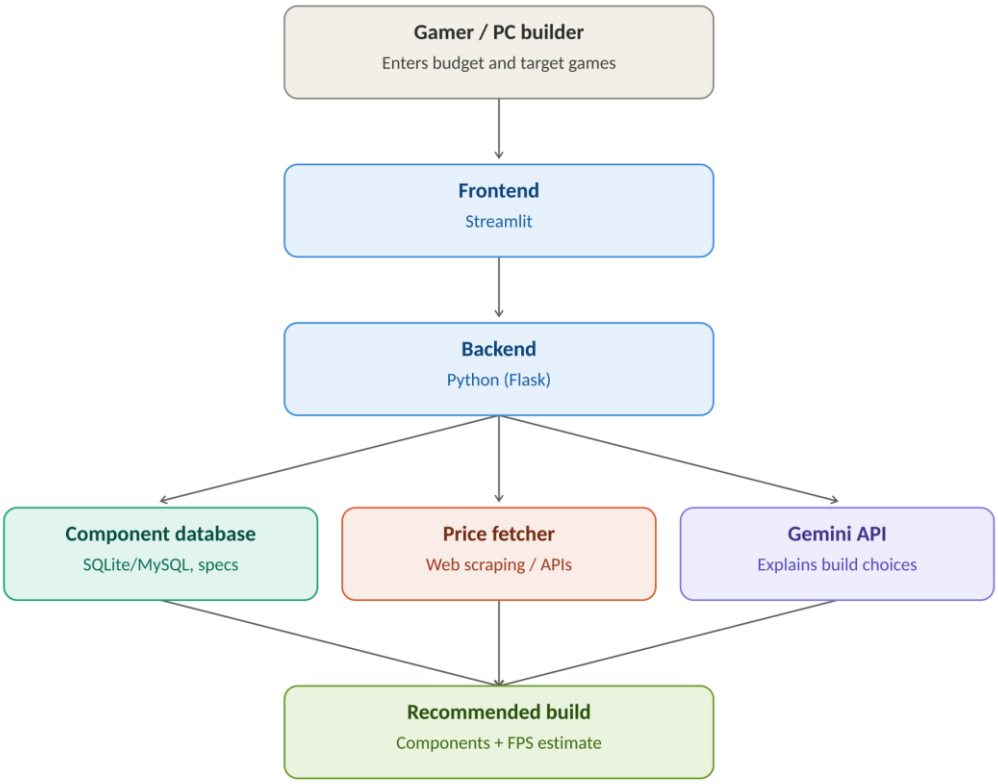
6.4 Communication Interface

The system shall communicate over standard HTTPS for all client-server and third-party API interactions.

7. System Architecture

7.1 Architecture Diagram

The diagram below shows the flow from user input through the frontend and backend to the component database, price fetcher, and Gemini API, converging into the final recommendation.



7.2 Frontend

The frontend is built using Streamlit, a Python framework that renders the build-configuration form, results screen, and admin dashboard without requiring separate HTML/CSS/JavaScript code.

7.3 Backend

The backend is built using Flask, a lightweight Python web framework that handles request routing, business logic (budget filtering, compatibility checks, upgrade logic), and orchestrates calls to the database and external APIs.

7.4 Database

SQLite is used during development for its simplicity and zero-configuration setup; the schema is designed to be portable to MySQL for a production-style deployment if needed.

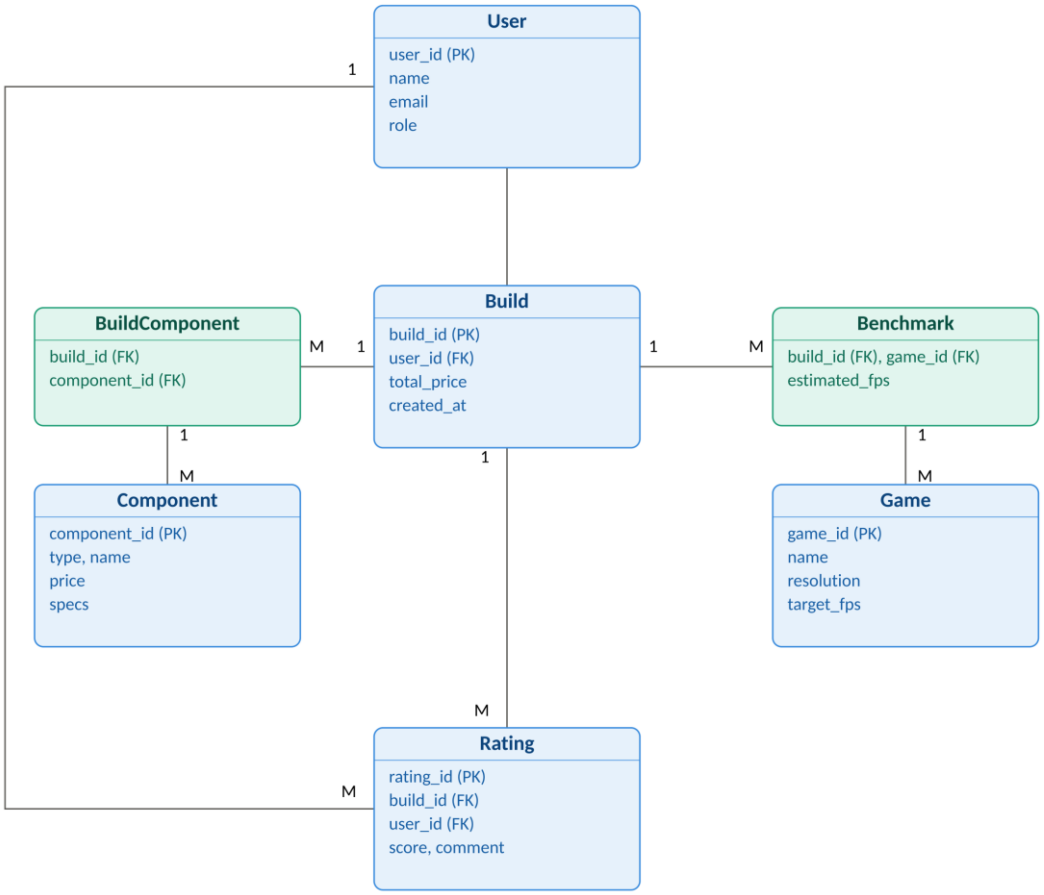
7.5 External APIs

The Gemini API is used to generate plain-language build explanations. Live component pricing is retrieved via retailer APIs where available, or through Python-based web scraping (BeautifulSoup/requests) where no public API exists.

8. Database Design

8.1 ER Diagram

The diagram below shows the entities, keys, and one-to-many relationships used to design the BuildAdvisor database, including the associative entities BuildComponent and Benchmark that resolve the many-to-many relationships between Build–Component and Build–Game respectively.



8.2 Data Dictionary

Entity	Attribute	Type	Description
User	user_id	Integer (PK)	Unique identifier for a user
User	name	Varchar	Display name
User	email	Varchar	Contact email
User	role	Varchar	Gamer, PC owner, or Admin
Component	component_id	Integer (PK)	Unique identifier for a component
Component	type	Varchar	CPU, GPU, RAM, etc.
Component	price	Decimal	Current listed price
Component	specs	Text	Technical specifications
Build	build_id	Integer (PK)	Unique identifier for a build
Build	user_id	Integer (FK)	Owner of the build
Build	total_price	Decimal	Sum of component prices
Rating	rating_id	Integer (PK)	Unique identifier for a rating
Rating	score	Integer	Community rating value
Game	game_id	Integer (PK)	Unique identifier for a game

Entity	Attribute	Type	Description
Benchmark	estimated_fps	Integer	Estimated FPS for a build/game pair

8.3 Database Tables

Table	Purpose
User	Stores gamer, PC owner, and Admin accounts
Component	Stores hardware parts available for recommendation
Build	Stores a saved or generated build configuration
BuildComponent	Associative table linking builds to their components
Game	Stores supported games and their resolution/FPS presets
Benchmark	Associative table storing estimated FPS per build/game pair
Rating	Stores community feedback on shared builds

8.4 Relationships

- One User creates many Builds (1:M)
- One User writes many Ratings (1:M)
- One Build receives many Ratings (1:M)
- Build and Component share a many-to-many relationship, resolved by BuildComponent
- Build and Game share a many-to-many relationship, resolved by Benchmark

9. Business Rules

9.1 Budget Rules

- The total cost of a recommended build must not exceed the user-specified budget.
- A minimum budget threshold applies below which no valid build can be generated; the system shall inform the user in this case.
- Budget tiers (entry-level, mid-range, high-end) are defined by fixed price boundaries maintained by the Admin.

9.2 Compatibility Rules

- CPU socket type must match the motherboard socket type.
- PSU wattage must exceed the combined TDP of the CPU and GPU by a safety margin.
- RAM type (DDR4/DDR5) must match what the motherboard supports.
- Selected components must fit the chosen case form factor.

9.3 FPS Rules

- Estimated FPS is derived from the closest matching component combination in the benchmark database.
- If no exact match exists, the nearest comparable combination is used and the estimate is marked as approximate.

9.4 Pricing Rules

- Displayed prices must reflect data fetched within a defined refresh window.
- If live price data is unavailable, the system shall fall back to the last known price and indicate the date it was recorded.

10. Error Handling

Scenario	System Response	User Impact
Invalid or missing budget input	Reject input and show an inline validation message	User is prompted to correct the input before proceeding
No compatible components found within budget	Show a message suggesting the nearest achievable budget	User can adjust budget or expectations
Pricing source unavailable	Fall back to last known price with a timestamp	User still sees a build, with a note on price freshness
Gemini API failure or timeout	Show the build without an AI explanation and a retry option	User still receives a usable recommendation
Database connection failure	Show a generic error and log the issue for the Admin	User is asked to retry later

11. Testing Requirements

11.1 Test Strategy

Testing will combine unit testing of individual functions (budget filtering, compatibility checks), integration testing of API calls (pricing source, Gemini API, database), system testing of the end-to-end build-generation flow, and informal user acceptance testing with a small group of gamers.

11.2 Test Cases

ID	Description	Input	Expected Output
TC01	Generate build within budget	Budget: 70000, Game: Valorant	Build total ≤ 70000, FPS estimate shown
TC02	Reject invalid budget	Budget: -500	Validation error shown, no build generated
TC03	Detect incompatible components	Mismatched CPU/motherboard sockets	Build rejected or component substituted
TC04	Fallback pricing	Pricing API unavailable	Last known price with timestamp shown
TC05	AI explanation returned	Valid build generated	Plain-language explanation displayed
TC06	Upgrade suggestion	Current specs + extra budget	Prioritized upgrade component suggested
TC07	Save and share build	Valid generated build	Shareable link generated
TC08	Admin adds component	New component details	Component appears in future recommendations

11.3 Acceptance Criteria

- A generated build never exceeds the specified budget.
- All recommended builds pass the compatibility checks defined in Section 9.2.
- An AI explanation is shown for at least 95% of successfully generated builds.
- The Admin can add, update, and remove components without developer intervention.

12. Security and Privacy

- Admin functions require authentication before database changes are allowed.
- All user-submitted input (ratings, saved build names) is validated and sanitized before storage to prevent injection attacks.
- Only minimal personal data (name, email, role) is stored; no payment or purchase data is collected.
- All communication with external APIs and between client and server uses HTTPS.

13. Limitations

- Live pricing depends on third-party sources and may not always be fully real-time or accurate.
- FPS estimates are limited to component combinations present in the benchmark database.
- AI-generated explanations may occasionally be generic if the Gemini API returns a low-detail response.
- The system only recommends builds; it does not process real purchases or payments.

14. Future Enhancements

- A mobile app version for on-the-go build planning.
- Direct purchase or affiliate links to retailers.
- User accounts with persistent saved-build history across devices.
- Support for more games and an expanded benchmark database.
- Real-time stock availability alongside pricing.

15. Requirements Traceability Matrix

Req. ID	Description	Source Section	Test Case
FR-1	Budget-based build generation	4.1	TC01, TC02
FR-2	Game-specific FPS targeting	4.2	TC01
FR-3	Component compatibility checking	4.3	TC03
FR-4	Live price comparison	4.4	TC04
FR-5	Upgrade path suggestions	4.5	TC06
FR-6	AI-generated build explanation	4.6	TC05
FR-7	Save, share, and rate builds	4.7	TC07
FR-8	Admin database management	4.8	TC08

16. Appendix

16.1 Glossary

Term	Description
Bottleneck	The component limiting overall system performance in a build
Tier	A budget category: entry-level, mid-range, or high-end
Benchmark	A recorded performance measurement (FPS) for a component combination in a specific game
Associative entity	A database entity used to resolve a many-to-many relationship between two other entities

16.2 References

- Google AI for Developers – Gemini API documentation
- Flask official documentation (flask.palletsprojects.com)
- Streamlit official documentation (docs.streamlit.io)
- Manufacturer specification sheets for CPU/GPU/motherboard compatibility data